Project on Caesar Cipher Encryption & Decryption Tool

Abstract

The Caesar Cipher Encryption and Decryption Tool is a Python-based application developed to demonstrate the principles of classical cryptography. The Caesar Cipher is one of the earliest and simplest encryption techniques, where each letter in the plaintext is shifted by a fixed number of positions in the alphabet to produce ciphertext.

The tool allows users to securely encrypt and decrypt text messages using a user-defined shift key. It provides an easy-to-use interface that accepts text input and performs both encryption and decryption while preserving spaces, punctuation, and letter cases. The project helps users understand the basic concepts of symmetric encryption, substitution ciphers, and secure communication.

It has been implemented using Python due to its simplicity, readability, and extensive support for string manipulation. The application is lightweight, platform-independent, and suitable for educational purposes. Although the Caesar Cipher is not secure against modern cryptographic attacks, it serves as an excellent introduction to encryption techniques and programming logic.

The project demonstrates the practical implementation of algorithms, user interaction, and modular programming while strengthening understanding of cybersecurity fundamentals and Python programming.

Introduction

Cryptography is the science of securing information by converting readable data into an unreadable format. Encryption protects sensitive information during communication and storage. One of the oldest encryption methods is the Caesar Cipher, developed by Julius Caesar to secure military communications.

This project develops a Caesar Cipher Encryption and Decryption Tool using Python. The application enables users to encrypt plain text into cipher text and decrypt cipher text back into readable text using a shift key. It provides practical experience in implementing cryptographic algorithms while enhancing Python programming skills.

Problem Statement

Sensitive information transmitted over communication channels can be intercepted by unauthorized users. Beginners often find modern encryption algorithms difficult to understand. Therefore, there is a need for a simple encryption tool that demonstrates the basic principles of cryptography using an easy-to-understand algorithm.

Objectives

- Develop a Python application for text encryption and decryption.
- Implement the Caesar Cipher algorithm.
- Allow users to specify a custom shift key.
- Preserve uppercase, lowercase, spaces, and symbols.
- Demonstrate the fundamentals of cryptography.
- Improve programming and problem-solving skills.

Scope

The project includes:

- Text encryption
- Text decryption
- User-defined shift key
- Command Line Interface (CLI)
- Validation of user inputs

Project boundaries:

- Supports English alphabet only.
- Does not provide modern cryptographic security.
- No online communication or cloud storage.

Literature Review

The Caesar Cipher is one of the earliest substitution ciphers introduced by Julius Caesar around 58 BC. It replaces each alphabet character with another character shifted by a fixed number of positions.

Existing encryption systems include:

Algorithm	Type	Security
Caesar Cipher	Substitution	Very Low
Vigenère Cipher	Polyalphabetic	Medium
AES	Symmetric	Very High
RSA	Asymmetric	Very High

Python provides excellent string handling capabilities that simplify implementing encryption algorithms.

Previous educational projects commonly use Caesar Cipher to introduce students to:

- Cryptography
- Algorithms
- String manipulation
- Python programming

Methodology

Step 1

Study the Caesar Cipher algorithm.

Step 2

Design program flow.

Step 3

Develop encryption function.

Step 4

Develop decryption function.

Step 5

Validate user input.

Step 6

Test with multiple shift values.

Step 7

Fix bugs and optimize code.

System Requirements

Hardware

- Intel Core i3 or above
- 4 GB RAM
- 500 MB storage
- Keyboard and Monitor

Software

- Windows/Linux/macOS
- Python 3.10+
- VS Code or PyCharm
- Command Prompt/Terminal

System Design

Flowchart

Start

|

Enter Text & Shift

|

Choose Encrypt/Decrypt

|

Process Each Character

|

Is Alphabet Character?

/ \

Yes No

| |

Shift Character Keep Same

| |

----Next Character----

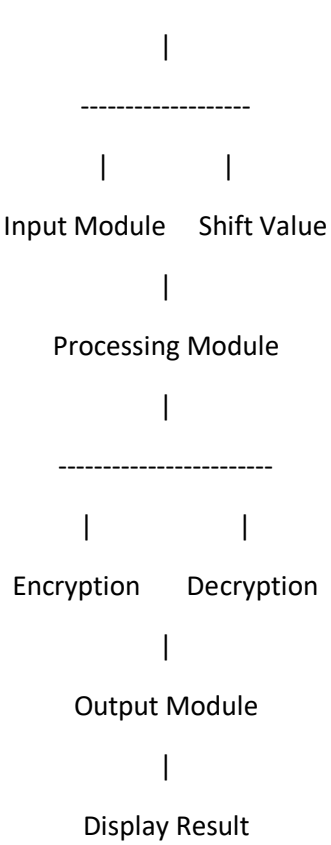
|

Display Result

|

End

Module Diagram



Technology Stack

Component	Technology
Programming Language	Python
IDE	VS Code / PyCharm
GUI (Optional)	Tkinter
Database	Not Required
Version Control	Git (Optional)
Operating System	Windows/Linux/macOS

Implementation

The project consists of two main functions:

Encryption Function

- Reads plaintext.
- Converts letters according to shift.
- Preserves symbols.
- Displays ciphertext.

Decryption Function

- Reads ciphertext.
- Applies reverse shift.
- Displays original text.

Modules Description

User Input Module

- Accepts plaintext.
- Accepts shift value.
- Validates input.

Encryption Module

- Applies Caesar Cipher.
- Converts plaintext to ciphertext.

Decryption Module

- Reverses encryption.
- Produces original text.

Output Module

- Displays encrypted message.
- Displays decrypted message.

Shahabuddin Shaikh

Testing

Functional Testing

Test Case	Input	Expected Result	Status
Encrypt text	HELLO	KHOOR	Pass
Decrypt text	KHOOR	HELLO	Pass
Shift 0	HELLO	HELLO	Pass
Symbols	HI!	KL!	Pass
Lowercase	hello	khoor	Pass

Challenges Faced

- Handling uppercase and lowercase letters separately.
- Managing alphabet wrap-around (Z to A).
- Preserving spaces and punctuation.
- Validating shift values.
- Supporting negative shifts.

Limitations

- Only supports English alphabets.
- Not secure for real-world communication.
- Vulnerable to brute-force attacks.
- No file encryption support.
- No database integration.

Future Enhancements

- Develop a graphical user interface (GUI) using Tkinter or PyQt.
- Add file encryption and decryption.
- Support multiple classical ciphers (Playfair, Rail Fence).
- Add password-based authentication.
- Maintain encryption history in a database.
- Create a web-based version using Flask or Django.

Shahabuddin Shaikh

Program –

```
from cryptography.fernet import Fernet
import codecs

# ---------------- Caesar Cipher ---------------- #

def caesar_encrypt(text, shift):
    result = ""

    for char in text:
        if char.isalpha():
            start = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char)-start+shift)%26+start)
        else:
            result += char

    return result



def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)



# ---------------- Vigenere Cipher ---------------- #

def vigenere_encrypt(text, key):
    result = ""
    key = key.upper()
    key_index = 0
```

```
    for char in text:
        if char.isalpha():
            shift = ord(key[key_index % len(key)]) - 65
            start = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char)-start+shift)%26+start)
            key_index += 1
        else:
            result += char

    return result


def vigenere_decrypt(text, key):
    result = ""
    key = key.upper()
    key_index = 0

    for char in text:
        if char.isalpha():
            shift = ord(key[key_index % len(key)]) - 65
            start = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char)-start-shift)%26+start)
            key_index += 1
        else:
            result += char

    return result
```

Shahabuddin Shaikh

```
# ---------- Atbash Cipher ---------- #

def atbash(text):
    result = ""

    for c in text:
        if c.isupper():
            result += chr(90 - (ord(c)-65))
        elif c.islower():
            result += chr(122 - (ord(c)-97))
        else:
            result += c

    return result



# ---------- ROT13 ---------- #

def rot13(text):
    return codecs.encode(text, 'rot_13')



# ---------- XOR Cipher ---------- #

def xor_encrypt(text, key):
    return ''.join(chr(ord(c)^key) for c in text)



def xor_decrypt(text, key):
    return ''.join(chr(ord(c)^key) for c in text)
```

Shahabuddin Shaikh

```
# ---------------- Fernet Encryption ---------------- #

fernet_key = Fernet.generate_key()
cipher = Fernet(fernet_key)

def fernet_encrypt(text):
    return cipher.encrypt(text.encode()).decode()

def fernet_decrypt(text):
    return cipher.decrypt(text.encode()).decode()



# ---------------- Main Menu ---------------- #

while True:

    print("\n========== Encryption Tool ==========")
    print("1. Caesar Cipher")
    print("2. Vigenere Cipher")
    print("3. Atbash Cipher")
    print("4. ROT13")
    print("5. XOR Cipher")
    print("6. Fernet (AES Based)")
    print("7. Exit")

    choice = input("\nChoose Algorithm: ")

    if choice == "7":
        break
```

```
    text = input("Enter Text: ")

    if choice == "1":
        shift = int(input("Shift: "))
        enc = caesar_encrypt(text, shift)
        print("Encrypted :", enc)
        print("Decrypted :", caesar_decrypt(enc, shift))

    elif choice == "2":
        key = input("Key: ")
        enc = vigenere_encrypt(text, key)
        print("Encrypted :", enc)
        print("Decrypted :", vigenere_decrypt(enc, key))

    elif choice == "3":
        enc = atbash(text)
        print("Encrypted :", enc)
        print("Decrypted :", atbash(enc))

    elif choice == "4":
        enc = rot13(text)
        print("Encrypted :", enc)
        print("Decrypted :", rot13(enc))

    elif choice == "5":
        key = int(input("Numeric Key: "))
        enc = xor_encrypt(text, key)
        print("Encrypted :", enc)
        print("Decrypted :", xor_decrypt(enc, key))

    elif choice == "6":
```

```
        enc = fernet_encrypt(text)
        print("Encrypted :", enc)
        print("Decrypted :", fernet_decrypt(enc))

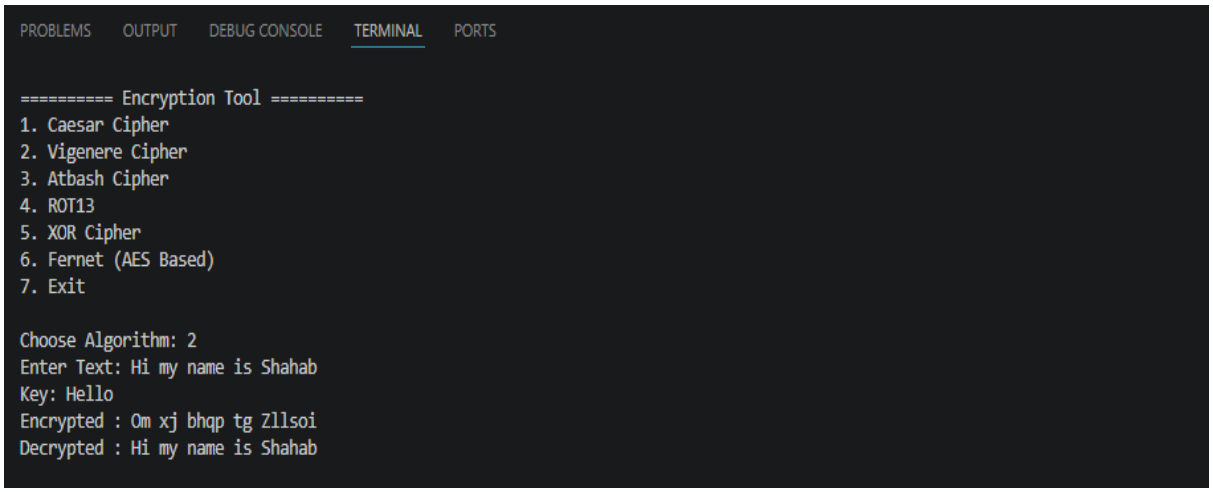
    else:
        print("Invalid Choice")
```

Results and Output

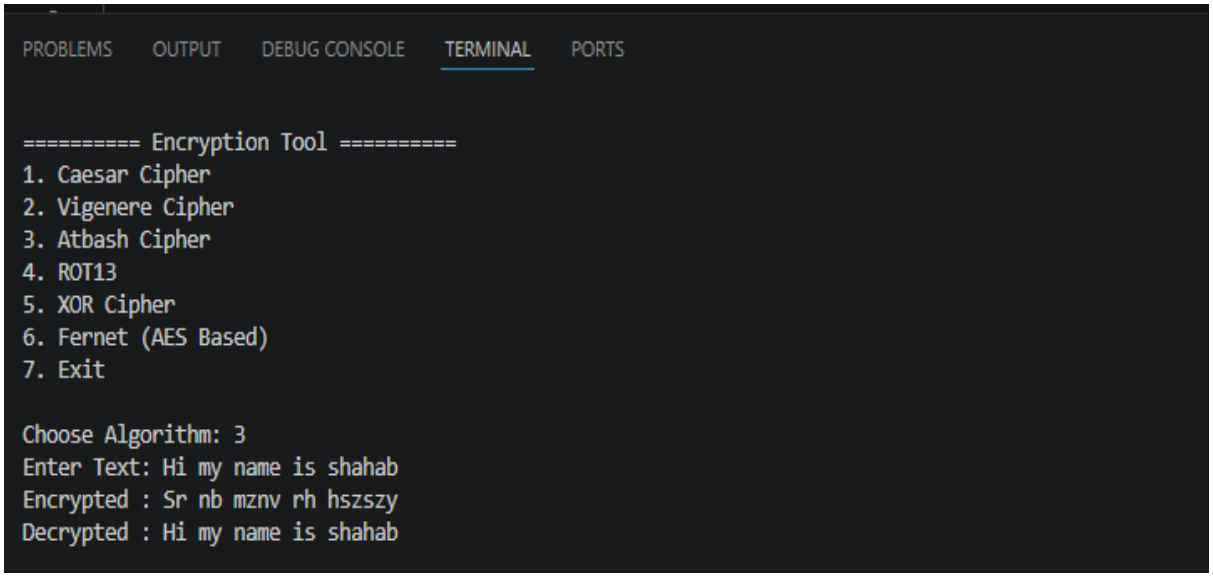
Caesar Cipher -



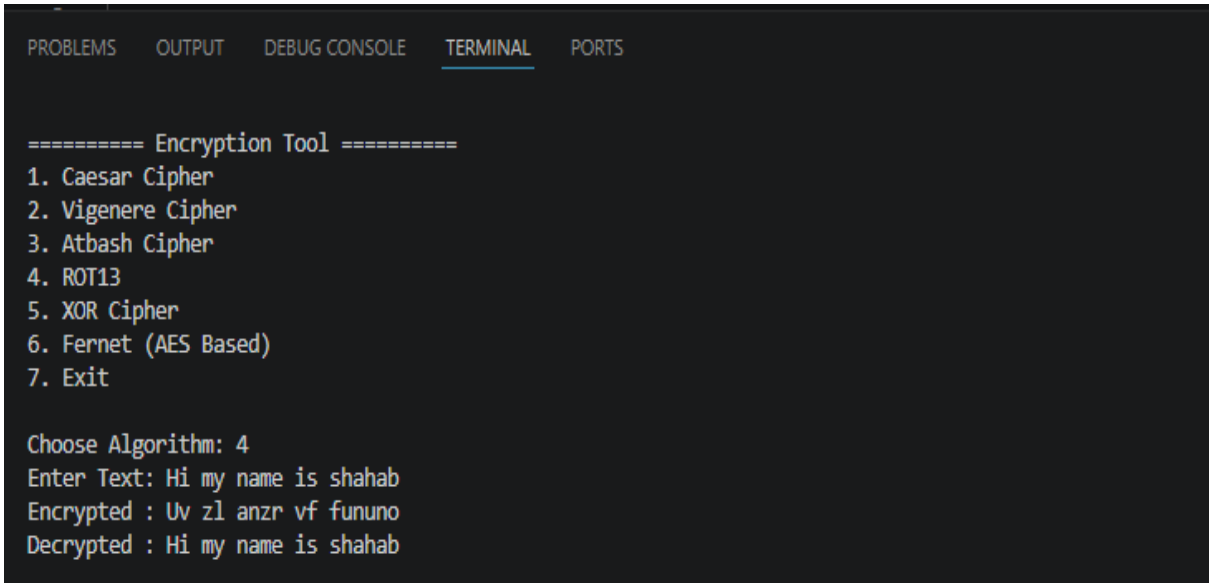
Vigenere Cipher -



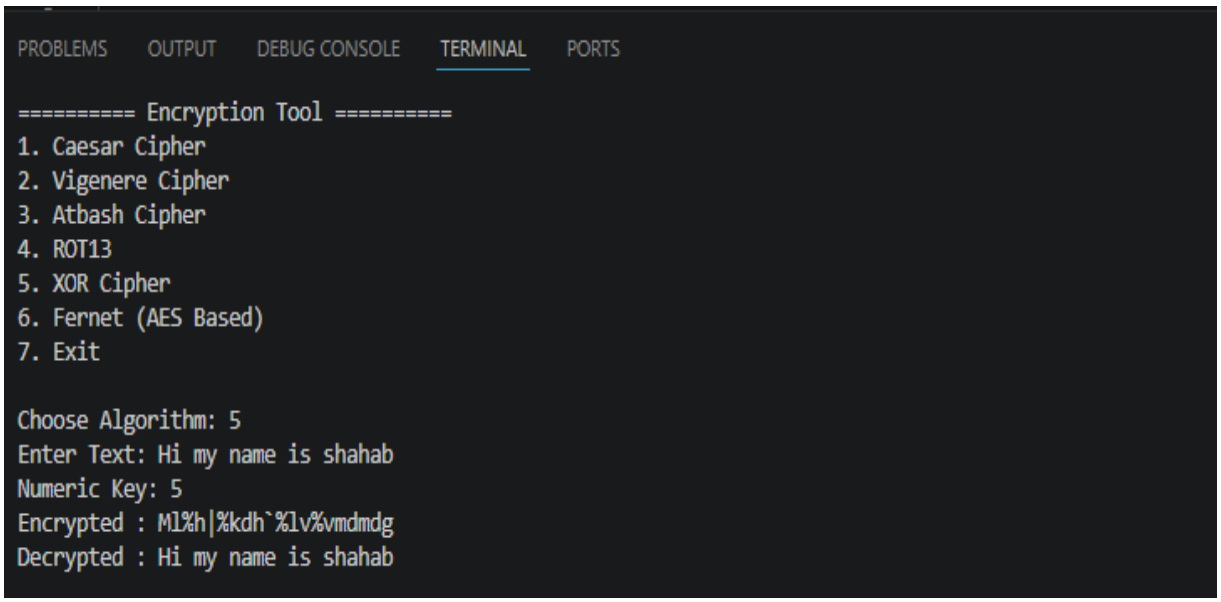
Atbash Cipher –



ROT13 -



XOR Cipher –



Fernet (AES Based) -



Conclusion

The Caesar Cipher Encryption and Decryption Tool successfully demonstrates the working principles of one of the oldest encryption techniques using Python. The project provides practical experience in implementing algorithms, string manipulation, modular programming, and user interaction.

While the Caesar Cipher is not suitable for modern secure communication due to its simplicity, it serves as an effective educational tool for understanding the fundamentals of cryptography.

Through this project, valuable skills were gained in Python development, debugging, testing, and software design. Future enhancements such as a graphical interface, file encryption, and stronger cryptographic algorithms can extend the project's functionality and educational value.